```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load the image in grayscale
img = cv2.imread('/content/landscape.jpg', cv2.IMREAD_GRAYSCALE)

# Check whether image is loaded
if img is None:
    print("Image not found. Please upload landscape.jpg")
else:
    print("Image loaded successfully!")
    print("Image Size:", img.shape)

# Display original image
plt.figure(figsize=(8, 6))
plt.imshow(img, cmap='gray')
plt.title('Original Grayscale Image')
plt.axis('off')
plt.show()
```

```
Image loaded successfully!
Image Size: (2667, 4000)
```




```
# Convert image to float32
img_float = np.float32(img)
```

```
# Compute Discrete Fourier Transform
dft = cv2.dft(img_float, flags=cv2.DFT_COMPLEX_OUTPUT)

print("Discrete Fourier Transform (DFT) calculated successfully.")
print("DFT Shape:", dft.shape)
```

```
Discrete Fourier Transform (DFT) calculated successfully.
DFT Shape: (2667, 4000, 2)
```

```
# Shift zero-frequency component to the center
dft_shift = np.fft.fftshift(dft)

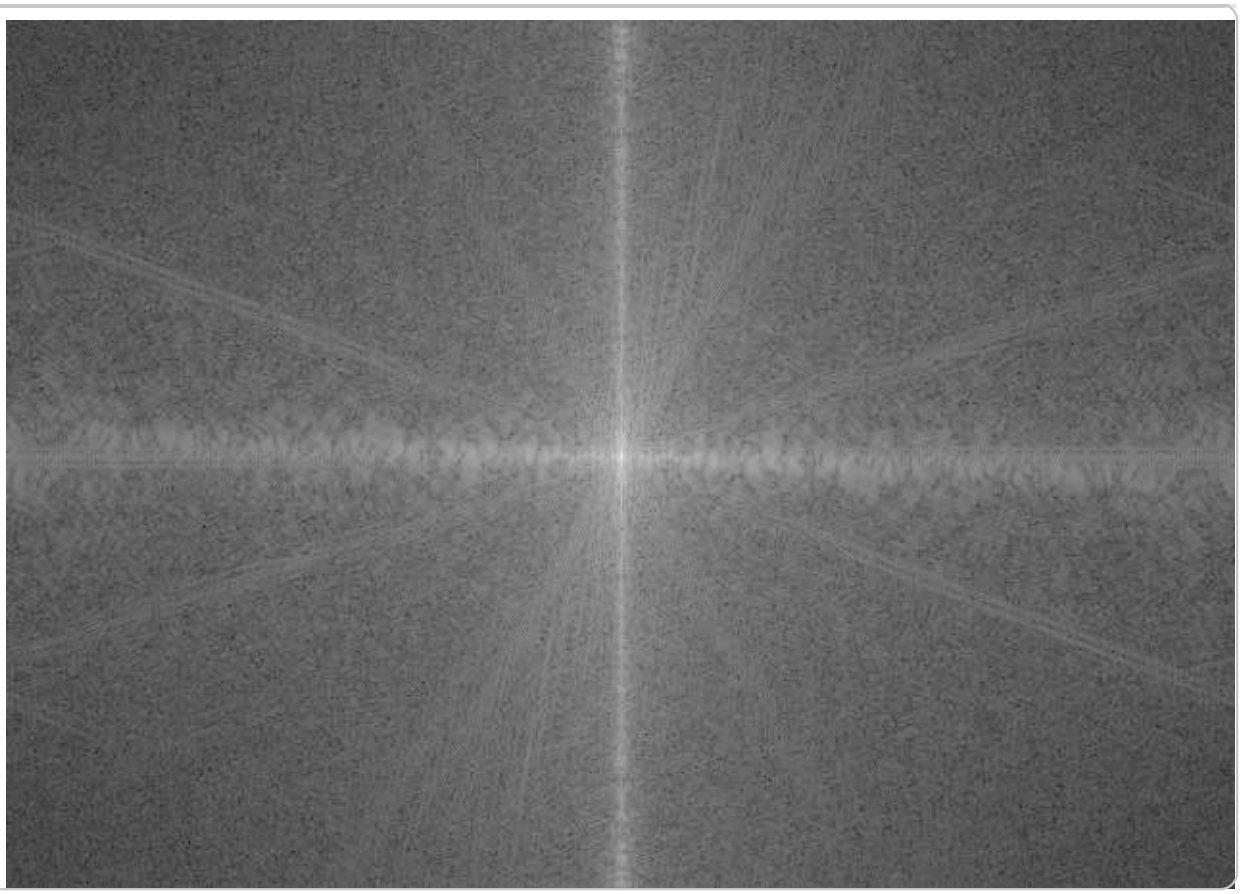
print("Zero-frequency component shifted to the center successfully.")
```

```
Zero-frequency component shifted to the center successfully.
```

```
# Calculate magnitude spectrum
magnitude_spectrum = cv2.magnitude(
    dft_shift[:, :, 0],
    dft_shift[:, :, 1]
)

# Apply logarithmic scale for better visualization
magnitude_spectrum = 20 * np.log(magnitude_spectrum + 1)

# Display magnitude spectrum
plt.figure(figsize=(8, 6))
plt.imshow(magnitude_spectrum, cmap='gray')
plt.title('Magnitude Spectrum')
plt.axis('off')
plt.show()
```



```
# Get image dimensions
rows, cols = img.shape

# Find center of image
crow, ccol = rows // 2, cols // 2

# Create Low-Pass Filter mask
mask_low = np.zeros((rows, cols, 2), np.float32)

# Radius of filter
radius = 50

# Keep low-frequency components around the center
cv2.circle(mask_low, (ccol, crow), radius, (1, 1), -1)

# Apply Low-Pass Filter
filtered_dft_low = dft_shift * mask_low

# Shift frequency back
f_ishift_low = np.fft.ifftshift(filtered_dft_low)

# Apply Inverse DFT
img_low = cv2.idft(f_ishift_low)

# Calculate magnitude
img_low = cv2.magnitude(
    img_low[:, :, 0],
```

```
    img_low[:, :, 1]
)

# Normalize image
img_low = cv2.normalize(
    img_low,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

# Convert to uint8
img_low = np.uint8(img_low)

# Display result
plt.figure(figsize=(8, 6))
plt.imshow(img_low, cmap='gray')
plt.title('Low-Pass Filtered Image')
plt.axis('off')
plt.show()
```



```
# Create High-Pass Filter mask
mask_high = np.ones((rows, cols, 2), np.float32)

# Remove low-frequency components from the center
cv2.circle(mask_high, (ccol, crow), radius, (0, 0), -1)
```

```
# Apply High-Pass Filter
filtered_dft_high = dft_shift * mask_high

# Shift frequency back
f_ishift_high = np.fft.ifftshift(filtered_dft_high)

# Apply Inverse DFT
img_high = cv2.idft(f_ishift_high)

# Calculate magnitude
img_high = cv2.magnitude(
    img_high[:, :, 0],
    img_high[:, :, 1]
)

# Normalize image
img_high = cv2.normalize(
    img_high,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

# Convert to uint8
img_high = np.uint8(img_high)

# Display result
plt.figure(figsize=(8, 6))
plt.imshow(img_high, cmap='gray')
plt.title('High-Pass Filtered Image')
plt.axis('off')
plt.show()
```




```
# Inverse Fourier Transform for Low-Pass Filtered Image
low_idft = cv2.idft(f_ishift_low)
low_result = cv2.magnitude(
    low_idft[:, :, 0],
    low_idft[:, :, 1]
)

# Inverse Fourier Transform for High-Pass Filtered Image
high_idft = cv2.idft(f_ishift_high)
high_result = cv2.magnitude(
    high_idft[:, :, 0],
    high_idft[:, :, 1]
)

print("Inverse Fourier Transform (IDFT) completed successfully.")
```

```
Inverse Fourier Transform (IDFT) completed successfully.
```

```
plt.figure(figsize=(14, 10))

# Original Image
plt.subplot(2, 2, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

# Magnitude Spectrum
```

```
plt.subplot(2, 2, 2)
plt.imshow(magnitude_spectrum, cmap='gray')
plt.title('Magnitude Spectrum')
plt.axis('off')

# Low-Pass Filtered Image
plt.subplot(2, 2, 3)
plt.imshow(img_low, cmap='gray')
plt.title('Low-Pass Filtered Image')
plt.axis('off')

# High-Pass Filtered Image
plt.subplot(2, 2, 4)
plt.imshow(img_high, cmap='gray')
plt.title('High-Pass Filtered Image')
plt.axis('off')

plt.tight_layout()
plt.show()
```

